

Échelle d'évaluation standard : B (% de réussite compris entre 50 et 75%)	Échelle d'évaluation pondérée : B (% de réussite compris entre 50 et 75%)
--	--

Initial FISA A3 - Sécurité et techniques de développement web - I

Échelle d'évaluation standard : B (% de réussite compris entre 50 et 75%)
--

Question 1

Question à réponse unique

Dans une architecture monolithique, quel est le principal inconvénient lors du passage à l'échelle (scaling) ?

Réponses correctes

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☐	Non	Chaque module doit être déployé sur une infrastructure distincte.
B	☒	☒	Non	Toute l'application doit être répliquée pour absorber la charge.
C	☐	☐	Non	Les échanges internes nécessitent des appels réseau systématiques.
D	☐	☐	Non	Les composants utilisent des bases de données indépendantes.

Question 2

Question à réponses multiples

Parmi les caractéristiques suivantes, lesquelles sont des avantages reconnus d'une architecture microservices ?

Réponses partiellement correctes

2 discordances

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☒	☒	Non	Déploiement indépendant de chaque service.
B	☐	☐	Non	Réduction systématique de la latence réseau entre composants.
C	☒	☒	Non	Possibilité d'utiliser des technologies différentes pour chaque service.
D	☒	☐	Oui (+1)	Complexité opérationnelle réduite par rapport à un monolithe.
E	☐	☒	Oui (+1)	Isolation des pannes : la défaillance d'un service n'entraîne pas obligatoirement l'arrêt de l'ensemble.

Question 3

Question à réponse unique

Qu'est-ce qu'une architecture orientée services (SOA) par rapport aux microservices ?

Réponses incorrectes

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☒	Oui (+1)	Les services sont généralement plus spécialisés et autonomes.
B	☒	☐	Oui (+1)	Les échanges reposent souvent sur un intermédiaire centralisé.
C	☐	☐	Non	Les communications utilisent exclusivement le protocole HTTP.
D	☐	☐	Non	Les déploiements nécessitent un environnement partagé unique.

Question 4

Question à réponses multiples

Lors de la représentation d'une architecture logicielle, quels diagrammes UML sont couramment utilisés pour décrire les composants et leurs interactions ?

Réponses partiellement correctes1 discordance

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☒	☒	Non	Diagramme de composants UML.
B	☐	☐	Non	Diagramme de Gantt.
C	☒	☐	Oui (+1)	Diagramme de contexte.
D	☒	☒	Non	Diagramme de séquence UML.
E	☐	☐	Non	Diagramme de classes UML pour modéliser les flux réseau.

Question 5

Question à réponse unique

La scalabilité horizontale consiste à :

Réponses incorrectes

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☐	Non	Augmenter les ressources matérielles d'une instance existante.
B	☒	☐	Oui (+1)	Répartir la charge entre plusieurs instances d'un même service.
C	☐	☒	Oui (+1)	Réduire les dépendances entre les composants applicatifs.
D	☐	☐	Non	Optimiser les algorithmes les plus consommateurs en ressources.

Question 6

Question à réponses multiples

Parmi les métriques suivantes, lesquelles sont couramment utilisées pour mesurer la disponibilité (availability) d'un système ?

Réponses partiellement correctes1 discordance

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☒	☐	Oui (+1)	SLA (Service Level Agreement) exprimé en pourcentage de temps de disponibilité.
B	☒	☒	Non	MTBF (Mean Time Between Failures).
C	☐	☐	Non	Le nombre de lignes de code du service.
D	☒	☒	Non	MTTR (Mean Time To Repair).
E	☐	☐	Non	La consommation mémoire moyenne.

Considérez le code Express.js suivant :

```
app.post('/users', async (req, res) => {
  const user = await db.createUser(req.body);
  res.send(user);
});
```

Quelle bonne pratique REST est violée dans cette route ?

Réponses incorrectes				
	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☐	Non	La réponse devrait inclure systématiquement un en-tête Cache-Control.
B	☒	☐	Oui (+1)	La création d'une ressource devrait renvoyer le statut HTTP 201 Created.
C	☐	☐	Non	La route devrait utiliser PUT au lieu de POST pour créer une ressource.
D	☐	☒	Oui (+1)	La réponse devrait obligatoirement contenir un identifiant dans l'en-tête Authorization.

Quelles bonnes pratiques sont recommandées pour le déploiement d'une API REST Node.js en production ?

Réponses partiellement correctes				1 discordance
	Réponse attendue	Réponse saisie	Réponse discordante	
A	☒	☐	Oui (+1)	Utiliser un gestionnaire de processus comme PM2 pour assurer le redémarrage automatique.
B	☐	☐	Non	Désactiver HTTPS pour réduire la latence des connexions.
C	☒	☒	Non	Mettre en place une gestion centralisée des erreurs (middleware d'erreur Express).
D	☒	☒	Non	Définir les variables sensibles (clés API, secrets) via des variables d'environnement et non dans le code source.
E	☐	☐	Non	Activer le mode verbose des logs en production pour faciliter le débogage.

Quel est le rôle du middleware suivant dans une application Express ?

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Erreur interne' });
});
```

Réponses correctes

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☐	Non	Intercepter toutes les requêtes entrantes pour les journaliser.
B	☒	☒	Non	Gérer les erreurs non capturées remontées via next(err) dans les routes.
C	☐	☐	Non	Servir des fichiers statiques depuis un répertoire public.
D	☐	☐	Non	Vérifier l'authentification de chaque requête.

Dans React, quel hook permet d'exécuter un effet de bord (appel API, abonnement) après le rendu d'un composant fonctionnel, avec nettoyage lors du démontage ?

Réponses correctes

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☐	Non	useState
B	☒	☒	Non	useEffect
C	☐	☐	Non	useContext
D	☐	☐	Non	useReducer

Question 11

Question à réponses multiples

Quelles phases font partie du cycle de vie d'un composant React basé sur les classes ?

Réponses correctes

0 discordance

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☒	☒	Non	componentDidMount
B	☐	☐	Non	componentWillRender
C	☒	☒	Non	componentDidUpdate
D	☒	☒	Non	componentWillUnmount
E	☐	☐	Non	componentFetched

Question 12

Question à réponse unique

Considérez ce composant React :

```
const [count, setCount] = useState(0); useEffect(() => {
  document.title = `Compteur : ${count}`;
}, [count]);
```

Quel est le comportement de ce useEffect ?

Réponses correctes

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☐	Non	L'effet s'exécute uniquement lors du premier rendu du composant.
B	☒	☒	Non	L'effet s'exécute après chaque rendu où la valeur de count a changé.
C	☐	☐	Non	L'effet s'exécute après chaque rendu, quelle que soit la valeur de count.
D	☐	☐	Non	L'effet ne s'exécute jamais car le tableau de dépendances est fourni.

Question 13

Question à réponses multiples

Quels éléments doivent typiquement figurer dans la modélisation d'une application frontend (ex. wireframes, maquettes fonctionnelles) ?

Réponses partiellement correctes

1 discordance

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☒	☒	Non	L'arborescence des vues/pages et la navigation entre elles.
B	☐	☐	Non	Les détails d'implémentation des algorithmes de tri.
C	☒	☒	Non	Les composants réutilisables et leur hiérarchie.
D	☒	☐	Oui (+1)	Les interactions utilisateur (événements, transitions d'état).
E	☐	☐	Non	La structure de la base de données relationnelle.

Question 14

Question à réponse unique

Dans un flux OAuth 2.0 de type Authorization Code, quel acteur délivre le access_token à l'application cliente ?

Réponses correctes

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☐	Non	Le client OAuth après authentification de l'utilisateur.
B	☐	☐	Non	Le serveur de ressources après validation des permissions.
C	☒	☒	Non	Le serveur d'autorisation après validation de la demande du client.
D	☐	☐	Non	Le fournisseur d'identité après résolution des rôles applicatifs.

Parmi les affirmations suivantes sur les JSON Web Tokens (JWT), lesquelles sont correctes ?

Réponses partiellement correctes			2 discordances	
	Réponse attendue	Réponse saisie	Réponse discordante	
A	☒	☐	Oui (+1)	Un JWT est composé de trois parties encodées en Base64 : header, payload et signature.
B	☐	☒	Oui (+1)	Le payload d'un JWT est chiffré par défaut, rendant son contenu illisible sans clé.
C	☒	☒	Non	La signature permet de vérifier l'intégrité du token et l'identité de l'émetteur.
D	☐	☐	Non	Il est recommandé de stocker des données sensibles non chiffrées dans le payload.
E	☒	☒	Non	Un JWT peut avoir une durée de validité définie via le claim exp.

Quelle est la différence principale entre les protocoles OAuth 2.0 et OpenID Connect (OIDC) ?

Réponses correctes				
	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☐	Non	OAuth 2.0 est un protocole d'authentification ; OIDC est un protocole d'autorisation.
B	☒	☒	Non	OAuth 2.0 gère l'autorisation ; OIDC ajoute une couche d'authentification au-dessus d'OAuth 2.0.
C	☐	☐	Non	OIDC remplace complètement OAuth 2.0 et n'est pas rétrocompatible.
D	☐	☐	Non	OAuth 2.0 utilise exclusivement des certificats X.509 pour signer les tokens.

Question 17

Question à réponses multiples

Lors de la mise en place d'une authentification JWT dans une API Node.js, quelles pratiques sont recommandées ?

Réponses partiellement correctes

1 discordance

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☒	☐	Oui (+1)	Signer le token avec un secret fort ou une paire de clés asymétriques (RS256).
B	☒	☒	Non	Définir une durée de vie courte pour le access_token et utiliser un refresh_token.
C	☐	☐	Non	Stocker le JWT dans le localStorage pour le rendre accessible à tous les scripts de la page.
D	☒	☒	Non	Vérifier la signature et l'expiration du token à chaque requête protégée.
E	☒	☒	Non	Transmettre le token dans l'en-tête HTTP Authorization: Bearer <token>.

Question 18

Question à réponse unique

Dans Redux, quel est le rôle unique d'un reducer ?

Réponses correctes

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☐	Non	Transformer des actions asynchrones en appels réseau.
B	☒	☒	Non	Produire un nouvel état à partir de l'état courant et d'une action reçue.
C	☐	☐	Non	Distribuer les actions aux différents stores de l'application.
D	☐	☐	Non	Synchroniser automatiquement le store avec l'interface utilisateur.

Question 19

Question à réponses multiples

Quels avantages offre l'utilisation d'un store centralisé dans une application frontend ?

Réponses incorrectes

3 discordances

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☒	☐	Oui (+1)	Source unique de vérité pour l'état de l'application.
B	☐	☒	Oui (+1)	Facilite le débogage via des outils comme Redux DevTools.
C	☒	☐	Oui (+1)	Élimine totalement le besoin de composants avec un état local.
D	☒	☒	Non	Simplifie le partage d'état entre composants éloignés dans l'arbre.
E	☐	☐	Non	Garantit une synchronisation automatique avec la base de données serveur.

Question 20

Question à réponse unique

Dans une application React / Next.js utilisant un gestionnaire d'état, à quoi correspond le concept de données dérivées du state ?

```
export const useAuthStore = create((set, get) => ({
  user: null,
  token: null,
  isAuthenticated: () => !!get().token,
}));
```

Réponses correctes

	Réponse attendue	Réponse saisie	Réponse discordante	
A	☐	☐	Non	Des fonctions qui modifient directement le state de manière synchrone.
B	☒	☒	Non	Des valeurs calculées à partir du state, utilisées pour exposer des données dérivées.
C	☐	☐	Non	Des fonctions asynchrones utilisées pour effectuer des appels API.
D	☐	☐	Non	Des mécanismes d'abonnement aux changements d'état (listeners).